



# Tecnológico de Monterrey

**Campus Monterrey**

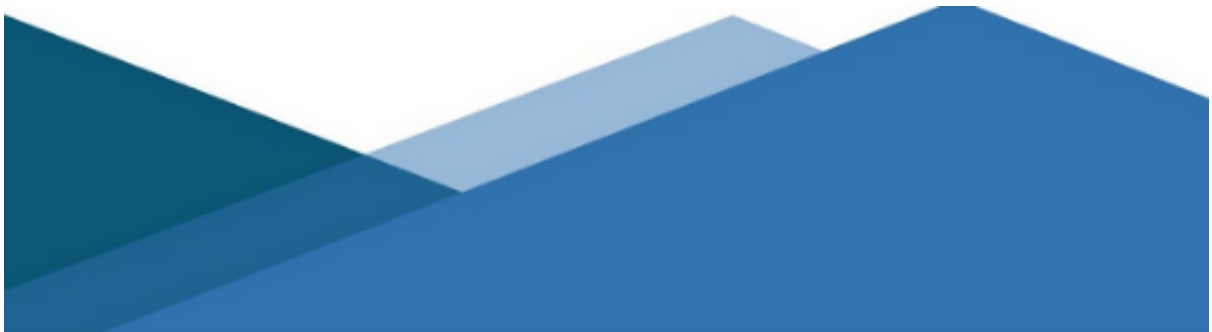
**Implementation of Computational Methods (Gpo 602)**

**Docente: Dr. Santiago Enrique Conant Pablos**

**Integrative Activity 6.2 Distributed Programming in Erlang**

**31 de Mayo del 2026  
(GRUPO 8)**

**Desiderio Iván Ortegón Mortón || A00840591  
Pablo Carrera Dollero || A00843410**



# Actividad 6.2: Programación Distribuida en Erlang - Reporte de Reflexión

## 1. Solución Propuesta y Código Desarrollado

Nuestra solución propuesta para el sistema distribuido de alquiler de taxis aprovecha el potente modelo de actores de Erlang y sus capacidades de paso de mensajes para crear una aplicación robusta y descentralizada. Dividimos el sistema en tres entidades principales los Travelers, los Taxis y el Taxi Dispatch Center. Cada entidad fue programada para operar como un proceso de Erlang independiente, capaz de ejecutarse en nodos distribuidos a lo largo de una red.

El código desarrollado encapsula el estado de cada entidad utilizando bucles recursivos de cola y registros de estado. El Centro de Despacho actúa como el registro central manteniendo listas de taxis disponibles, viajes activos y un historial de trayectos completados. Implementamos protocolos de comunicación tanto síncronos como asíncronos:

- Uso del operador ! para mensajes asíncronos de tipo dispara y olvida.
- Bloques recibe con tiempos de espera para las solicitudes síncronas.

Para garantizar la robustez del sistema, incorporamos el monitoreo de procesos. Si el proceso de un taxi falla o termina inesperadamente el Centro de Despacho detecta la señal 'down' y elimina de forma segura al taxi de la lista activa.

## 2. Operación del Sistema

El sistema opera a través de una coreografía fluida de intercambios de mensajes:

1. **Inicialización:** Se arranca el Centro de Despacho de Taxis, vinculándose a un nombre registrado para que otros nodos puedan localizarlo en la red sin necesidad de conocer su PID específico.
2. **Registro:** Los taxis se crean en sus respectivos nodos y envían un mensaje de registro al servidor central el cual los añade al grupo de unidades disponibles.
3. **Solicitud:** Cuando un viajero solicita un viaje, su proceso envía un mensaje al Centro de Despacho con su ID único y sus coordenadas geográficas.
4. **Asignación:** El Centro evalúa los taxis activos, calcula la distancia euclidiana para encontrar el vehículo disponible más cercano y le envía una solicitud a ese taxi.
5. **Confirmación:** Tras la aceptación del taxi, el Centro genera un ID de Viaje único, actualiza el estado del taxi a "ocupado" e informa al viajero.

Las salidas de la terminal muestran explícitamente este flujo imprimiendo trazas de "Sends:..." y "Receives:..." para seguir el ciclo de vida exacto de la solicitud a través de la red distribuida. Cuando el taxi completa el servicio notifica al nodo central el cual archiva el viaje y libera al taxi para futuras solicitudes.

### 3. La Parte Más Difícil

La parte mas difícil de este proyecto fue gestionar la sincronización de procesos y la consistencia del estado entre nodos distribuidos. Específicamente el manejo de las condiciones de carrera que ocurren cuando múltiples viajeros solicitan un taxi simultáneamente. Esto requirió una implementación sumamente meticulosa del bucle de manejo de mensajes en el Centro de Despacho.

Para asegurar que un mismo taxi no fuera asignado a dos viajeros diferentes en el mismo milisegundo, tuvimos que serializar estrictamente las actualizaciones de estado dentro del bloque receive del servidor central. Además configurar la conexión entre nodos (`net_kernel` y `net_adm:ping`) y asegurarse de que los firewalls permitieran la comunicación sin restricciones entre los nodos de Erlang requirió una cantidad considerable de soporte técnico y resolución de problemas.

### 4. La Parte Más Fácil

Por el contrario, la parte más sencilla de la implementación fue la definición de las funciones matemáticas y auxiliares como el cálculo de la distancia euclidiana entre el viajero y los taxis.

La coincidencia de patrones de Erlang también hizo que fuera increíblemente directo y limpio procesar los diferentes tipos de mensajes entrantes. Escribir los manejadores para las actualizaciones de estado simples como cuando un taxi pasa de "iniciado" a "completado" resultó muy intuitivo ya que la sintaxis de Erlang está diseñada de forma nativa para soportar este tipo de lógica de máquinas de estado.

### 5. Orden de Complejidad del Código Implementado

La complejidad temporal de nuestro sistema varía según la operación específica que se esté ejecutando:

- **Solicitud de Taxi (Lógica de Asignación):** La complejidad temporal es  $O(N)$ , donde  $N$  es el número de taxis disponibles en ese momento. Cuando un viajero pide un viaje, el Centro de Despacho debe iterar por toda la lista de taxis disponibles para calcular la distancia euclidiana de cada uno, ordenarlos y seleccionar el más cercano.
- **Registro de un Taxi / Finalización de un Viaje:** La complejidad es  $O(1)$ . Agregar un nuevo taxi al inicio de la lista de disponibles o registrar un viaje completado en el historial se realiza en tiempo constante, ya que las listas en Erlang manejan las inserciones en la cabeza en un tiempo de  $O(1)$ .

- **Paso de Mensajes:** Localizar el proceso registrado del centro y enviarle un mensaje opera en un tiempo de  $O(1)$  abstrayendo la latencia física de la red.

En general el sistema está altamente optimizado para inserciones y actualizaciones rápidas presentando un cuello de botella de búsqueda lineal únicamente durante la fase de asignación por distancia lo cual es perfectamente aceptable para el alcance de este prototipo.

Pruebas:

```
Erlang
{ok,<0.87.0>}
(central@Desiderio10)3> c(center).
center.erl:111:58: Warning: variable 'Origin' is unused
% 111| lists:foreach(fun({TripId, TaxiId, Traveler, Origin, Status}) ->
% |
center.erl:153:19: Warning: variable 'TripId' is unused
% 153| dispatch_proposal(TripId, Traveler, Origin, TravelerPid, [], AirportLocation, Taxis, ActiveTrips, CompletedTrips
% | TripCounter) ->
% |
center.erl:153:27: Warning: variable 'Traveler' is unused
% 153| dispatch_proposal(TripId, Traveler, Origin, TravelerPid, [], AirportLocation, Taxis, ActiveTrips, CompletedTrips
% | TripCounter) ->
% |
center.erl:153:37: Warning: variable 'Origin' is unused
% 153| dispatch_proposal(TripId, Traveler, Origin, TravelerPid, [], AirportLocation, Taxis, ActiveTrips, CompletedTrips
% | TripCounter) ->
% |
{ok,center}
(central@Desiderio10)4> c(taxi).
{ok,taxi}
(central@Desiderio10)5> c(traveler).
{ok,traveler}
(central@Desiderio10)6> center:open_center({10,10}).
Sends: opening dispatch center at {10,10}
{ok,<0.109.0>}
(central@Desiderio10)7> |
```

```
Erlang
Erlang/OTP 29 [erts-17.0] [source] [64-bit] [smp:12:12] [ds:12:12:10] [async-threads:1] [jit:ns]

Eshell V17.0 (press Ctrl+G to abort, type help(). for help)
1> cd("C:/Users/desid/Downloads/Integrative Activity 6.2 Distributed Programming in Erlang/Integrative-Activity-6.2-Distributed-Programming-in-Erlang/src").
c:/Users/desid/Downloads/Integrative Activity 6.2 Distributed Programming in Erlang/Integrative-Activity-6.2-Distributed-Programming-in-Erlang/src
ok
2> net_kernel:start([vehiculos, shortnames]).
{ok,<0.87.0>}
(vehiculos@Desiderio10)3> net_adm:ping('central@Desiderio10').
pong
(vehiculos@Desiderio10)4> |
```

```
Erlang
Erlang/OTP 29 [erts-17.0] [source] [64-bit] [smp:12:12] [ds:12:12:10] [async-threads:1] [jit:ns]

Eshell V17.0 (press Ctrl+G to abort, type help(). for help)
1> cd("C:/Users/desid/Downloads/Integrative Activity 6.2 Distributed Programming in Erlang/Integrative-Activity-6.2-Distributed-Programming-in-Erlang/src").
c:/Users/desid/Downloads/Integrative Activity 6.2 Distributed Programming in Erlang/Integrative-Activity-6.2-Distributed-Programming-in-Erlang/src
ok
2> net_kernel:start([pasajeros, shortnames]).
{ok,<0.87.0>}
(pasajeros@Desiderio10)3> net_adm:ping('central@Desiderio10').
pong
(pasajeros@Desiderio10)4> |
```

## Registrar el Taxi

```
Erlang
(center@Desiderio10)3> c(center).
center.erl:111:58: Warning: variable 'Origin' is unused
% 111 | lists:foreach(fun({TripId, TaxiId, Traveler, Origin, Status}) ->
% |
center.erl:153:19: Warning: variable 'TripId' is unused
% 153 | dispatch_proposal(TripId, Traveler, Origin, TravelerPid, [], AirportLocation, Taxis, ActiveTrips, CompletedTrips
% | TripCounter) ->
% |
center.erl:153:27: Warning: variable 'Traveler' is unused
% 153 | dispatch_proposal(TripId, Traveler, Origin, TravelerPid, [], AirportLocation, Taxis, ActiveTrips, CompletedTrips
% | , TripCounter) ->
% |
center.erl:153:37: Warning: variable 'Origin' is unused
% 153 | dispatch_proposal(TripId, Traveler, Origin, TravelerPid, [], AirportLocation, Taxis, ActiveTrips, CompletedTrips
% | , TripCounter) ->
% |

{ok,center}
(center@Desiderio10)4> c(taxi).
{ok,taxi}
(center@Desiderio10)5> c(traveler).
{ok,traveler}
(center@Desiderio10)6> center:open_center([10,10]).
Sends: opening dispatch center at {10,10}
{ok,<0.109.0>}
Receives: registration request from taxi_01 at {0,0}
(center@Desiderio10)7> |

Erlang
Erlang/OTP 29 [erts-17.0] [source] [64-bit] [smp:12:12] [ds:12:12:10] [async-threads:1] [jit:n]

Eshell V17.0 (press Ctrl+C to abort, type help(). for help)
1> cd("C:/Users/desid/Downloads/Integrative Activity 6.2 Distributed Programming in Erlang/Integrative-Activity-6.2-Distributed-Programming-in-Erlang/src").
c:/Users/desid/Downloads/Integrative Activity 6.2 Distributed Programming in Erlang/Integrative-Activity-6.2-Distributed-Programming-in-Erlang/src
ok
2> net_kernel:start([vehiculos, shortnames]).
{ok,<0.87.0>}
(vehiculos@Desiderio10)3> net_adm:ping('central@Desiderio10').
pong
(vehiculos@Desiderio10)4> taxi:register_taxi(taxi_01, {0,0}).
Sends: taxi_01 registering at location {0,0}
ok
(vehiculos@Desiderio10)5> |
```

El Pasajero pide el Taxi

|                                                                                                                                   |                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> Receives: taxi request from traveler_1 at {2,2} Sends: center proposes Trip 1 to taxi_01 (central@Desiderio10)13&gt; </pre> | <pre> pong (vehiculos@Desiderio10)10&gt; c(taxi). {ok, taxi} (vehiculos@Desiderio10)11&gt; taxi:register taxi( Sends: taxi_01 registering at location {0,0} ok Receives: trip proposal for Trip 1 from travel </pre> | <pre> (usuarios@Desiderio10)12&gt; net_adm:ping('central@Desiderio10'). pong (pasajeros@Desiderio10)4&gt; c(traveler). {ok, traveler} (pasajeros@Desiderio10)5&gt; traveler:request_taxi(traveler_1, {2,2}). Sends: traveler_1 requests taxi from {2,2} </pre> |
|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Terminal 3 (pasajero)

```
(pasajeros@Desiderio10)5> traveler:request_taxi(traveler_1, {2,2}).
Sends: traveler_1 requests taxi from {2,2}
```

## Terminal 2 (vehiculos)

```
Receives: trip proposal for Trip 1 from traveler_1 at {2,2}
```

terminal 1 (central)

```
Receives: taxi request from traveler_1 at {2,2}  
Sends: center proposes Trip 1 to taxi_01
```

Finalizar viaje

Terminal 2 (vehiculos)

```
(vehiculos@Desiderio10)6> taxi:accept_trip(taxi_01, 1).  
Sends: taxi_01 accepts Trip 1  
ok  
Receives: assignment confirmed for Trip 1  
(vehiculos@Desiderio10)7> taxi:service_started(taxi_01).  
Sends: taxi_01 notifies service started  
ok  
(vehiculos@Desiderio10)8> taxi:service_completed(taxi_01).  
Sends: taxi_01 notifies service completed  
ok  
(vehiculos@Desiderio10)9> |
```

terminal 1 (central)

```
Sends: center proposes Trip 1 to taxi_01  
Receives: trip acceptance from taxi_01 for Trip 1  
Receives: service started notification from taxi_01  
Receives: service completed notification from taxi_01
```

Terminal 3 (pasajero)

```
Receives: taxi request accepted. Assigned Taxi: taxi_01, Trip ID: 1  
{ok,{taxi_01,1}}
```

Historial de la central

(Terminal 1)

```
--- HISTORIAL DE VIAJES COMPLETADOS ---  
Trip ID: 1 | Taxi: taxi_01 | Pasajero: traveler_1 | Origen: {2,2}  
1  
(central@Desiderio10)6> center:close_center().  
Sends: closing dispatch center  
Receives: Shutting down center... terminating taxis  
ok
```

## Reflexión del Equipo

Esta actividad nos sirvió para aplicar de manera práctica los conceptos de programación distribuida. A diferencia de otros proyectos en los que los componentes comparten memoria o se ejecutan dentro de un mismo proceso en esta actividad cada entidad del sistema fue implementada como un actor independiente utilizando Erlang. Esto nos obligó a pensar en términos de comunicación mediante mensajes y coordinación entre procesos distribuidos lo que representó un reto importante y al mismo tiempo una oportunidad para comprender mejor el paradigma de concurrencia que ofrece este lenguaje.

Una de las partes más difíciles del proyecto fue diseñar correctamente el flujo de comunicación entre las entidades traveler, taxi y center. Aunque conceptualmente el sistema parece sencillo en la práctica fue necesario definir cuidadosamente qué mensajes se envían, quién los recibe y qué acciones debe ejecutar cada proceso al recibirlos. La lógica de aceptación y rechazo de viajes requirió una planificación detallada ya que el centro debía mantener información sobre las solicitudes pendientes y reaccionar adecuadamente cuando un taxi rechazaba una propuesta. También fue necesario implementar mecanismos para que el centro intentara automáticamente contactar al siguiente taxi disponible sin perder la información de la solicitud original.

Otro reto importante fue la administración del estado del sistema ya que cada proceso mantiene su propio estado interno fue necesario diseñar estructuras de datos adecuadas para almacenar la información de taxis activos, pasajeros registrados, viajes completados y solicitudes pendientes. Comprendimos que en Erlang el estado no se modifica directamente sino que se reconstruye y se pasa recursivamente a la siguiente iteración del ciclo del proceso. Este enfoque fue diferente a otros lenguajes que hemos utilizado anteriormente y requirió un periodo de adaptación.



Una de las partes más sencillas fue la implementación de la comunicación básica mediante mensajes utilizando el operador de envío de Erlang. Una vez comprendido el funcionamiento de los procesos y los bloques receive, fue mas fácil establecer la comunicación entre los distintos componentes del sistema. También resultó sencilla la implementación del cálculo de la distancia euclidiana para determinar cuál taxi se encontraba más cerca del pasajero. La dificultad real no estuvo en el cálculo matemático sino en integrarlo dentro de la lógica distribuida del sistema.

La incorporación de registros de trazabilidad mediante mensajes “Sends” y “Receives” fue especialmente útil durante el desarrollo gracias a estos logs fue posible observar en tiempo real cómo circulaban los mensajes entre los nodos y verificar que cada proceso estuviera respondiendo correctamente. además de facilitar la depuración estos registros permitieron comprender mejor el comportamiento interno del sistema distribuido y demostrar visualmente su funcionamiento.

Otro aprendizaje importante fue la utilización de pruebas unitarias mediante EUnit. Estas pruebas permitieron validar aspectos críticos del sistema como el cálculo correcto de la distancia euclidiana el incremento del contador de viajes y el registro adecuado de taxis dentro del estado del centro. La experiencia demostró la importancia de verificar automáticamente la funcionalidad de componentes clave para reducir errores durante el desarrollo.

Esta actividad nos permitió comprender de manera práctica los principios fundamentales de la programación distribuida utilizando Erlang. Aprendimos a diseñar sistemas basados en actores, coordinar procesos independientes mediante mensajes, administrar estados distribuidos y realizar pruebas de validación. Consideramos que el proyecto cumplió con los objetivos establecidos y nos proporcionó una experiencia valiosa para entender cómo funcionan los sistemas concurrentes y distribuidos en entornos reales.